

Fiche d'investigation de fonctionnalité

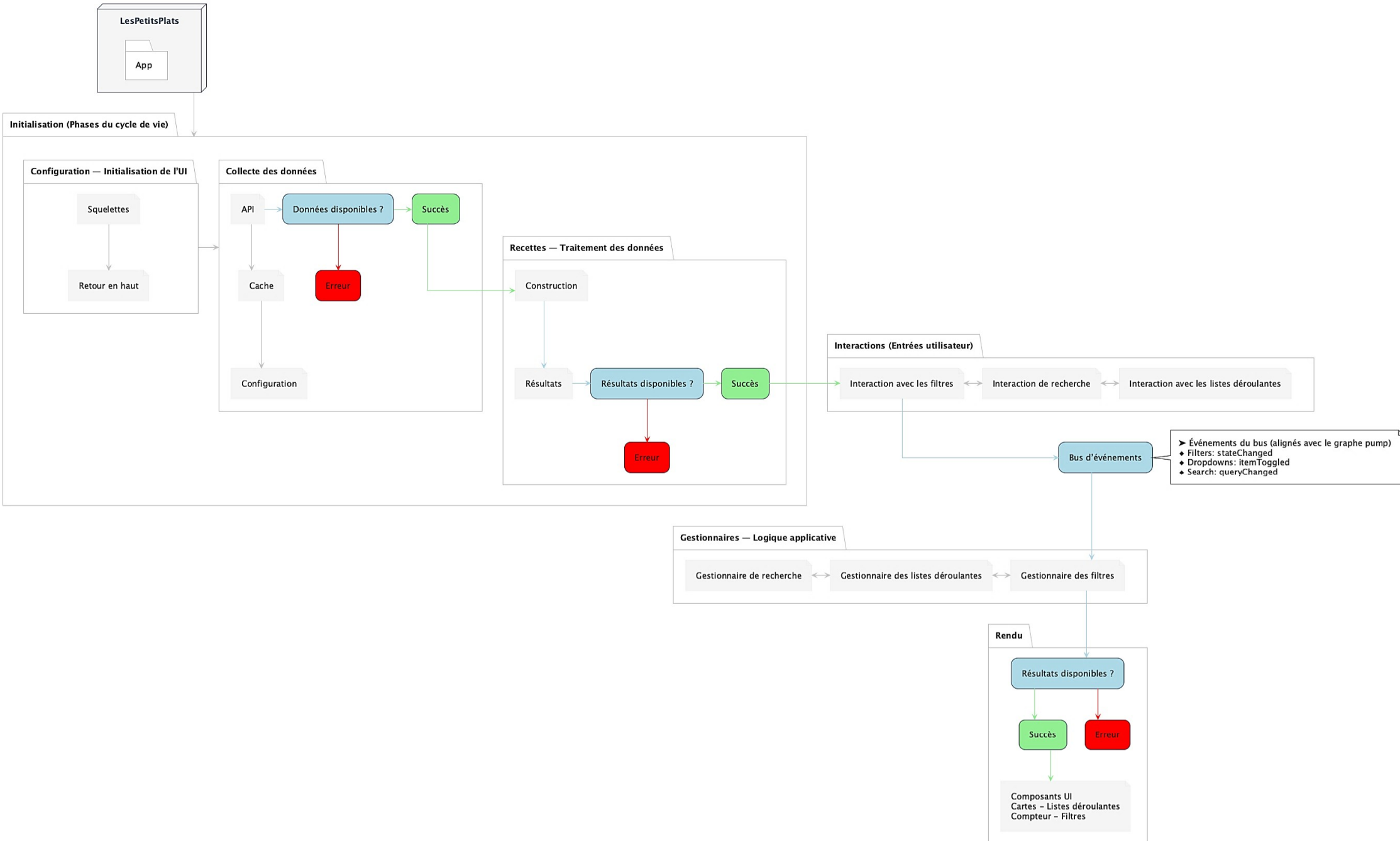
Fonctionnalité : Filtrer les recettes dans l'interface utilisateur		Fonctionnalité #1
Problématique : Pour se différencier des autres plateformes de recettes, nous souhaitons proposer un moteur de recherche, fluide/performant. L'objectif est d'afficher, en temps réel, les recettes correspondant à la saisie de l'utilisateur. Les filtres de la recherche avancée doivent s'adapter dynamiquement pour refléter uniquement les options compatibles avec les résultats courants.		
Option 1 : Utilisation des boucles natives Parcours manuel des recettes : dès qu'un élément (titre, ingrédient, description) correspond à la recherche, la recette est ajoutée à la liste finale.		
Avantages Implémentation simple Logique facile à comprendre		Inconvénients : Complexité accrue si les conditions deviennent nombreuses Maintenance moins aisée dans un projet collaboratif
Nombre de champs minimum à remplir: 1 Le champ principal de recherche est obligatoire pour déclencher le premier filtrage.		

Option 2 : Méthodes avancées de l'objet Array (filter(), some()) On filtre les recettes directement via les méthodes natives de tableaux filter() pour restreindre la liste ainsi que some() pour vérifier si un champ correspond à la recherche Seules les recettes qui correspondent réellement sont retournées et affichées.		
Avantages Code plus compact, lisible et expressif Performances améliorées, méthodes optimisées Rendu plus fluide pour l'utilisateur S'adapte mieux à des évolutions futures		Inconvénients Nécessite une bonne connaissance des méthodes tableaux Gestion des retours parfois plus abstraite Risque de créer des fonctions anonymes trop complexes Debug parfois moins évident
Nombre de champs minimum à remplir: 1 Le champ principal de recherche est obligatoire pour déclencher le premier filtrage.		

Solution retenue : Après analyse, nous retenons l'Option 2 — Programmation avancée avec les méthodes de l'objet Array.	
Raisons du choix : • Meilleure lisibilité et maintenabilité, performances supérieures (opérations plus rapides) • Code plus moderne et adapté à un moteur de recherche dynamique • Cohérent avec une architecture modulaire et collaborative	
Cette approche permet d'obtenir un moteur de recherche qui est Plus fluide, plus rapide plus agréable pour l'utilisateur final	

Les Petits Plats — Architecture de l'application

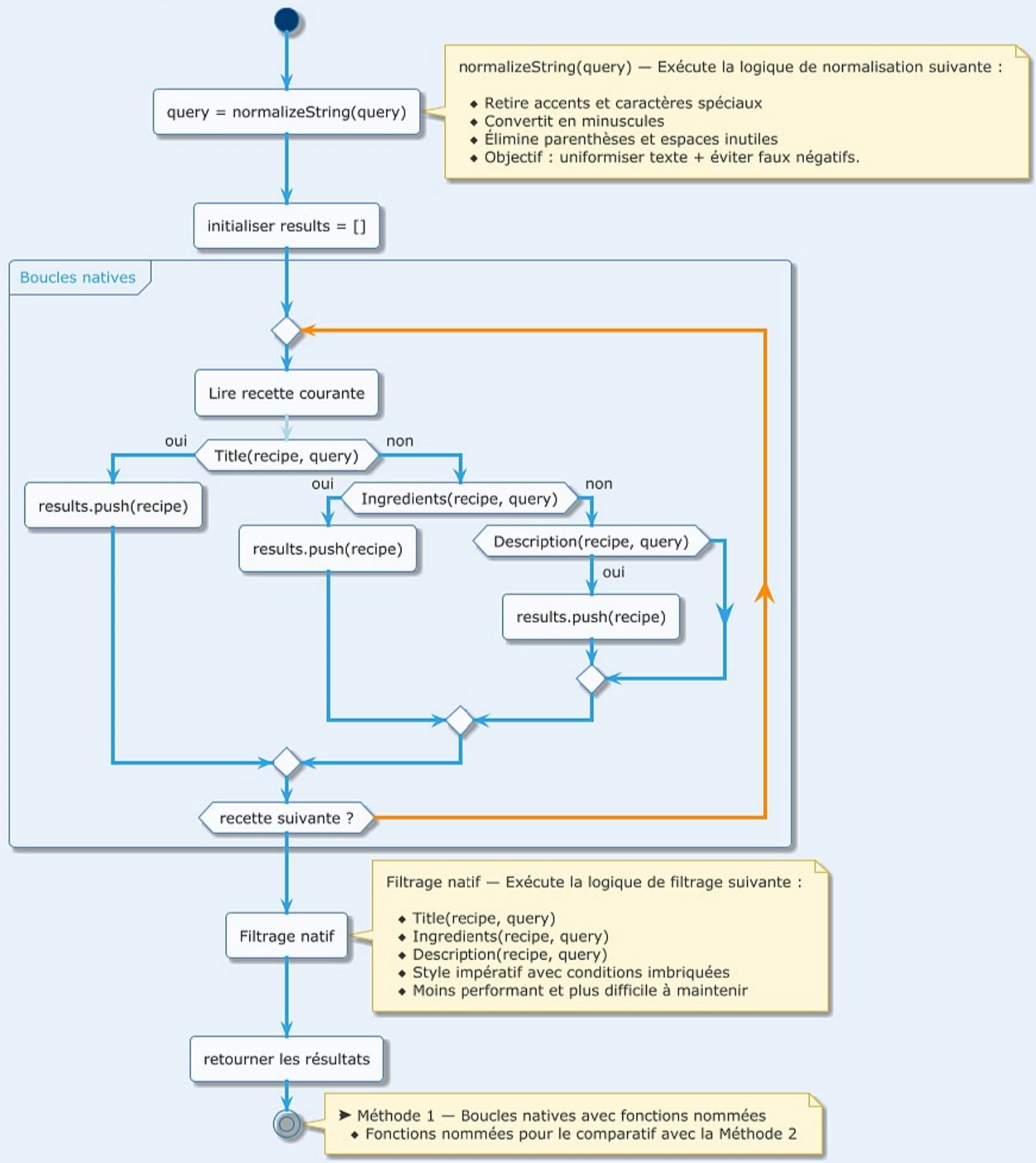
Vue structurelle et phases du cycle de vie



Les Petits Plats — Stratégie de filtrage (Méthode 1)

Approche impérative utilisant des boucles natives

Moteur de recherche



Les Petits Plats — Stratégie de filtrage (Méthode 2)

Approche déclarative utilisant filter(), some() et applyAllFilters()

Moteur de recherche

query = normalizeString(query)

normalizeString(query) — Exécute la logique de normalisation suivante :

- ◆ Retire accents et caractères spéciaux
- ◆ Convertit en minuscules
- ◆ Élimine parenthèses et espaces inutiles
- ◆ Objectif : uniformiser texte + éviter faux négatifs.

Filtrage fonctionnel

results = applyAllFilters(recipes, query)

applyAllFilters() — Exécute la logique de filtrage suivante :

- Normalisation :
 - ◆ Les champs utiles sont comparés via normalizeString().
- Conditions de correspondance :
 - ◆ matchTitle(recipe, query) : Vérifie si le titre contient la requête normalisée.
 - ◆ recipe.ingredients.some(matchIngredient) : Recherche un ingrédient correspondant
 - ◆ matchDescription(recipe, query) : Analyse la description
- Règle de validation :
 - ◆ La recette est retenue si AU MOINS UNE condition est vraie.

retourner les résultats

- Méthode 2 — Filtrage fonctionnel
 - ◆ Utilisation de Array.filter() et Array.some()
 - ◆ applyAll() comme logique centrale de filtrage

Les Petits Plats — Flux de données à l'exécution

Traitement des données piloté par événements et mises à jour de l'interface

